W7 Code Guide — Read the Model Behind the Function

The two notebooks use original NumPy/SciPy implementations from the course repository. The table links directly to the definitions at the fixed `2026-fall-w7` source reference. Notebook setup verifies byte hashes before importing downloaded code. Imported course functions are therefore inspectable without searching through a moving repository.

API	Exact source location	Purpose
`truth`	Definition	Known synthetic latent function; not given to the fitted models.
`make_data`	Definition	Return nested training observations and an independent fixed test grid.
`gaussian_nll`	Definition	Sum Gaussian negative log densities, including normalization constants.
`bernoulli_nll`	Definition	Stable sum binary cross entropy; logits are real, labels must be 0 or 1.
`poisson_nll`	Definition	Sum Poisson NLL. log_rate is eta; exp(eta) is the expected count.
`total_variance`	Definition	Finite mixture mean, within-component variance, between-component variance.
`BayesianLinear`	Definition	Exact Gaussian posterior conditional on fixed features and known noise.
`BayesianLinear.fit`	Method	features: (n,d), y: (n,); zero-mean isotropic Gaussian weight prior.
`BayesianLinear.predict`	Method	Return mean (m,), full cov_f (m,m), var_f and var_y (m,).
`rbf_kernel`	Definition	One-dimensional RBF covariance. amplitude is a function standard deviation.
`GaussianProcess`	Definition	Zero-mean RBF GP; exact conditional inference with fixed hyperparameters.
`GaussianProcess.fit`	Method	Store a Cholesky factor of K + noise**2 I; never invert K explicitly.
`GaussianProcess.predict`	Method	Joint latent-function posterior; new observation variance adds noise**2.
`TinyMLP`	Definition	Point-estimated one-hidden-layer tanh network: a minimal nonlinear predictor.
`TinyMLP.fit`	Method	Fit by L-BFGS; status records convergence, iterations and objective change.

API	Exact source location	Purpose
`TinyMLP.features`	Method	Frozen learned features (n,width+1), including an intercept column.
`TinyMLP.predict`	Method	Point predictions only; repeated calls are not posterior samples.
`prior_nn_functions`	Definition	Prior function draws; no data conditioning. Output weights scale as 1/sqrt(width).
`draw_functions`	Definition	Joint Gaussian function draws; preserve cross-input correlation.
`evaluate_predictions`	Definition	Observed-y RMSE, mean NLL, empirical pointwise 95% coverage and mean width.
`fit_prediction`	Definition	Fit on training observations only; return predictive dict and honest fit status.
`run_experiment`	Definition	Return figure, region metrics DataFrame, status, CSV. No test-based selection.
`likelihood_demo`	Definition	One observation: show how a real score becomes a valid distribution parameter.
`create_app`	Definition	Construct Gradio Blocks without starting a server; optional new-metric callback.

Data and array contracts

`make_data(n, noise, seed)` returns a dictionary. Training `x` and `y` have shape `(n,)`; the test-grid `xt`, independently noisy test response `yt`, and simulator truth `ft` have shape `(241,)`. `noise` is a standard deviation. Holding seed and noise fixed while increasing n preserves previous training pairs. The true function and test outcomes are for evaluation; no fit routine receives them.

`BayesianLinear.fit(features, y)` expects an `(n,d)` feature matrix and an `(n,)` response vector. Its prior is zero-mean Gaussian with covariance `prior_variance * I`. Include your own intercept column. `predict(features)` returns an `(m,)` mean, an `(m,m)` latent covariance, and `(m,)` latent and future-response variances. The latter adds `noise**2`.

`GaussianProcess.fit(x,y)` expects one-dimensional inputs. It forms the RBF training covariance plus observation-noise variance on the diagonal and stores a Cholesky factor. Kernel amplitude is a standard deviation. `length` and noise are fixed assumptions in this lesson, not values tuned against the test set.

`TinyMLP.fit` estimates one parameter vector. `features` returns frozen hidden features with an intercept column, while `predict` returns point predictions only. Its `status` records optimizer convergence, iterations, termination reason and objective reduction. A nonconverged optimization is not silently labeled successful. A last-layer Bayesian calculation is obtained by passing these frozen features to `BayesianLinear`; hidden-feature uncertainty is omitted.

`draw_functions(mean,covariance)` samples a joint Gaussian vector across the whole input grid. Sampling each grid value independently from its marginal would discard the covariance and would not draw from the same function posterior.

Loss conventions

The NLL helpers return **sums**, including distribution normalization constants. Gaussian NLL accepts variance, unlike model constructors' `noise` standard deviation. Bernoulli NLL accepts logits and binary labels. Poisson NLL accepts log-rates and nonnegative integer counts. Invalid support values raise an explicit error. `evaluate_predictions` divides Gaussian NLL by the number of observations to report mean NLL.

The MLP objective is mean squared error divided by two plus `regularization * squared_weight_norm / 2`. It is a point-fitting objective, not a full posterior computation. Optimization uses L-BFGS; hand-calculated loss examples remain independent of the optimizer.

Web-app contract

`run_experiment` returns a Matplotlib figure, a pandas DataFrame with four region rows, a model-status string, and a CSV path. `create_app` builds Gradio Blocks without starting a server. The notebook explicitly launches it at the end. A share address is temporary and works only while its runtime remains alive.

The optional `extra` callback receives a fresh metrics dictionary for each region and must return a dictionary of **new** metric names. A name collision raises an error rather than overwriting a reported metric. This gives you a small, theory-linked modification task without needing to know Gradio's internal classes.

Reproducibility and troubleshooting

- The notebooks are independent. Run setup even if you completed the other notebook.
- Pinned versions are NumPy 2.2.6, SciPy 1.16.3, Matplotlib 3.10.3, pandas 2.3.3 and Gradio 6.27.0. Setup prints the interpreter and versions.
- If setup changes an already imported package, restart the runtime once and run all again. Do not continue with a mixture of old imports and newly installed files.
- A source hash mismatch stops execution. Reopen the published notebook rather than deleting the check or importing a different revision.
- A failed temporary download can be retried. Successfully checked source files are reused on a subsequent run.
- Local non-Colab execution builds the app without starting a server by default. Set `LAUNCH_WEB_APP=True` in the final cell to open it locally.
- Apparent repeated-prediction agreement is expected for a deterministic fitted network. It does not validate a posterior sampler.

External library documentation

NumPy arrays, SciPy Cholesky solves, SciPy optimization, and Gradio Blocks describe the library interfaces. Course-definition links above identify the implementation actually used in this lesson.